23 Oct 2023

HW-10- due Friday

Exams: M, 30 Oct. 2023
F, 17 Nov. 2023

Projects: due after Thanksgiving
deliverable: a video
"proposal" due 6 Nov.

} Reminders

Independent Set

Input: A graph $G = (V, E)$
- V = vertices
- E = edges = (unordered) pairs of vertices

Output: the maximum size (cardinality) $W \subseteq V$ such that no two verts in W are connected. (aka - they're independent)

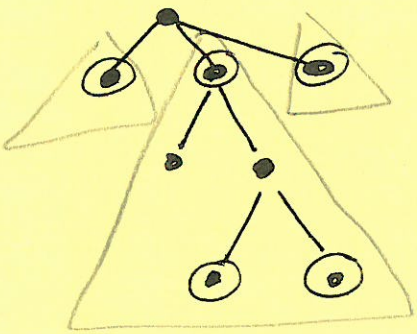
→ a "textbook" NP-Hard problem.

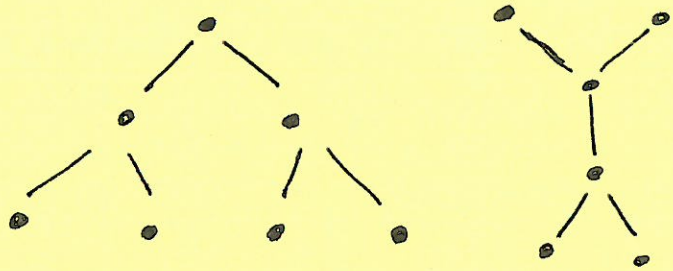
Let's come up w/ a DP for I.S. on Trees.

Def^n: A tree is a connected graph with no simple cycles. Can be rooted or not.

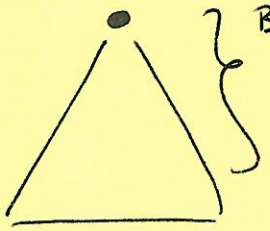
Hint: to start, let's assume the tree is rooted (you can assume binary tree if needed too)

Examples:





Hint:



① What if I know that I do not use the root?

② What if I know that I do use the root?

of v; this number can be very different from one vertex to the next. But we can turn the analysis around: Each vertex contributes a constant amount of time to its parent and its grandparent! Because each vertex has at most one parent and at most one grandparent, the algorithm runs in ***O(n) time***.

Here is the resulting dynamic programming algorithm. Yes, it's still recursive, because that's the most natural way to implement a post-order tree traversal.

```
TREEMIS(v):
  skipv ← 0
  for each child w of v
      skipv ← skipv + TREEMIS(w)
  keepv ← 1
  for each grandchild x of v
      keepv ← keepv + x.MIS
  v.MIS ← max{keepv, skipv}
  return v.MIS
```

We can derive an even simpler linear-time algorithm by defining two separate functions over the nodes of T:

- Let $MISyes(v)$ denote the size of the largest independent set of the subtree rooted at v that *includes* v.
- Let $MISno(v)$ denote the size of the largest independent set of the subtree rooted at v that *excludes* v.

Again, we need to compute $\max\{MISyes(r), MISno(r)\}$, where r is the root of T. The first two functions satisfy the following mutual recurrence:

$$MISyes(v) = 1 + \sum_{w \downarrow v} MISno(w)$$

$$MISno(v) = \sum_{w \downarrow v} \max\{MISyes(w), MISno(w)\}$$

Again, we can memoize these functions into the tree itself, by defining two new fields for each vertex. A straightforward post-order tree traversal evaluates both functions at every node in ***O(n) time***. The following algorithm not only memoizes both function values at v, it also returns the larger of those two values.

```
TREEMIS2(v):
  v.MISno ← 0
  v.MISyes ← 1
  for each child w of v
      v.MISno ← v.MISno + TREEMIS2(w)
      v.MISyes ← v.MISyes + w.MISno
  return max{v.MISyes, v.MISno}
```